

Cheatsheet Esame Pratico — Sistemi Operativi

Forme generalizzate dei task ricorrenti (C & Python)

Indice

1 Regole d'esame: Esercizio 0 e consegna	1
1.1 Esercizio 0 — "Se copiate vi cacciamo"	1
1.2 Vincolo FONDAMENTALE su come eseguire comandi	1
1.3 Esercizio finale — consegna	1
2 Librerie / header: cosa includere e perché	1
2.1 C — header POSIX	1
2.2 Python — moduli	2
3 Iterare i file di una directory	2
3.1 C — iterazione manuale + ricorsione (<code>opendir/readdir</code>)	2
3.2 C — visita automatica dell'albero (<code>nftw</code>)	2
3.3 Python — <code>os.walk</code> (ricorsivo) e <code>os.scandir</code> (un livello)	2
4 Controllare il tipo / i metadati di un file	2
4.1 C — <code>stat</code> vs <code>lstat</code>	2
4.2 C — riconoscere il contenuto dai "magic bytes"	3
4.3 Python — tipo e permessi	3
5 Hard link, symlink, inode	3
5.1 Concetti	3
5.2 C — creare link	3
5.3 C — trovare hard link / symlink di un file (pattern d'esame classico)	3
5.4 Python — link	3
6 Creare / spostare / copiare directory e file	4
6.1 C — <code>mkdir</code> , <code>rename</code> (spostamento atomico), copia ricorsiva	4
6.2 Python — alto livello	4
7 Leggere / scrivere file e contenuto	4
7.1 C — leggere un intero file in memoria	4
7.2 C — digest SHA1 (OpenSSL, compila <code>-lssl -lcrypto</code>)	4
7.3 Python — testo, encoding, digest	4
8 Argomenti, processi, exec	4
8.1 C — parametri e usage	4
8.2 C — <code>fork</code> + <code>exec</code> + <code>wait</code>	4
8.3 C — leggere args da <code>/proc</code> (cmdline è separato da <code>\0</code>)	5
9 Comunicazione tra processi (IPC)	5
9.1 C — FIFO / named pipe	5
9.2 C — segnali real-time (passano un valore, sono in coda)	5
9.3 C — trasmettere bit a bit con <code>SIGUSR1/SIGUSR2</code> + <code>ack</code> (variante d'esame)	5
9.4 C — contare segnali con <code>signalfd</code> (variante d'esame)	5
10 inotify — reagire a eventi del filesystem	6
11 Pattern Python ricorrenti	6
11.1 Raggruppare file per criterio + creare cartelle/symlink	6
11.2 Ordinare i file per profondità / nome / inode	6
11.3 Tabella inode con profondità limitata (<code>scandir</code> ricorsivo)	6
11.4 Aggiungere testo come commento in base al linguaggio (per suffisso)	6
11.5 Confronto contenuto file / eliminare duplicati di <code>f</code> (<code>dremcont</code>)	6
12 Errori tipici da NON fare (checklist mentale)	7

1 Regole d'esame: Esercizio 0 e consegna

Compagno in OGNI prova. Falli subito, valgono punti / squalifica.

1.1 Esercizio 0 — "Se copiate vi cacciamo"

Rendere la home inaccessibile ad altri (no lettura, no esecuzione). Tenere una sola dir in /public col proprio username e permessi 700.

```
chmod 700 ~ # home solo per me
mkdir -p /public/nome.cognome
chmod 700 /public/nome.cognome
```

1.2 Vincolo FONDAMENTALE su come eseguire comandi

VIETATO: system, popen o simili; **vietato** fare exec di sh -c. In Python usare subprocess *senza* shell, **mai** os.system/os.spawn.

```
// C: SEMPRE fork + execv/execvp (mai system())
pid_t p=fork();
if(p==0){ execvp(args[0], args); perror("execvp"); _exit(127); }
else waitpid(p,NULL,0);
```

```
import subprocess
subprocess.run(["ls", "-l", "/tmp"]) # lista di argomenti, shell=False (default)
# MAI: os.system(...), os.popen(...), shell=True
```

1.3 Esercizio finale — consegna

- Mail a renzo@cs.unibo.it, oggetto **esatto** PROVAPRATICA, allegati *separati*.
- Nomi file con il proprio cognome. Rinominare i .py con suffisso diverso (es. .pytxt).
- I file consegnati devono essere **privi di commenti**.
- Mostrare/allegare l'output di sha1sum di ogni file consegnato. Niente link a cloud.

```
sha1sum *.c *.pytxt # lasciare l'output a video nel terminale
```

2 Librerie / header: cosa includere e perché

2.1 C — header POSIX

<stdio.h>	printf, fopen, fread, fgets, snprintf, perror.
<stdlib.h>	malloc, realloc, free, exit, atoi, realpath, EXIT_*.
<string.h>	strcmp, strcpy, strlen, strdup, strchr, strcspn, memcmp, memmem.
<unistd.h>	read, write, close, readlink, symlink, link, unlink, fork, exec*, getcwd, getpid.
<fcntl.h>	open, flag O_RDONLY/O_WRONLY/O_CREAT.
<dirent.h>	opendir, readdir, closedir, struct dirent (iterazione manuale di una directory).
<sys/stat.h>	stat, lstat, mkdir, mkfifo, macro S_ISREG/S_ISDIR/S_ISLNK, bit permessi S_IXUSR.
<sys/types.h>	tipi: ino_t, off_t, pid_t, ssize_t.
<ftw.h>	nftw (visita ricorsiva automatica dell'albero). Serve #define _XOPEN_SOURCE 500 <i>prima</i> degli include.
<sys/wait.h>	wait, waitpid, macro WEXITSTATUS.
<sys/inotify.h>	inotify_init, inotify_add_watch, struct inotify_event.
<signal.h>	sigaction, sigqueue, sigemptyset, union sigval, SIGRTMIN/SIGRTMAX. Serve #define _POSIX_C_SOURCE 200809L.
<limits.h>	PATH_MAX (dimensione buffer per i path).
<errno.h>	errno, costanti EEXIST ecc. (gestione errori fine).
<openssl/sha.h>	SHA1/SHA256. Compila con -lssl -lcrypto.
<time.h>	time_t (confronto st_mtime).

Feature-test macro (riga 1, prima di tutto): _XOPEN_SOURCE 500 per nftw; _GNU_SOURCE per memmem/FTW_ACTIONRETVAL; _POSIX_C_SOURCE 200809L per i segnali real-time.

2.2 Python — moduli

os	os.walk, os.scandir, os.stat/os.lstat, os.symlink, os.makedirs, os.path.*, os.sep.
sys	sys.argv (parametri), sys.exit.
stat	stat.filemode (stringa tipo -rwxr-xr-x), stat.S_ISDIR ecc.
shutil	shutil.copy, shutil.move, shutil.rmtree (operazioni ad alto livello).
hashlib	hashlib.sha1()/sha256() per digest file.
collections	defaultdict per raggruppare.
dataclasses	@dataclass per record semplici.

3 Iterare i file di una directory

3.1 C — iterazione manuale + ricorsione (opendir/readdir)

Usa quando devi controllare tu la ricorsione, costruire path, o serve flessibilità.

```
void scan_dir(const char *path) {
    DIR *dir = opendir(path);
    if (!dir) { perror("opendir"); return; }

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
        // SEMPRE saltare "." e ".." o ricorsione infinita
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0) continue;

        // costruire il path completo: dir + "/" + nome
        char full[PATH_MAX];
        snprintf(full, sizeof(full), "%s/%s", path, entry->d_name);

        struct stat st;
        if (lstat(full, &st) == -1) continue; // lstat NON segue i symlink

        if (S_ISDIR(st.st_mode)) scan_dir(full); // ricorsione
        else if (S_ISREG(st.st_mode)) printf("%s\n", full);
    }
    closedir(dir);
}
```

3.2 C — visita automatica dell'albero (nftw)

Usa quando basta "visita tutto e per ogni file fai X". Niente ricorsione manuale.

```
#define _XOPEN_SOURCE 500
#include <ftw.h>
// typeflag: FTW_F=file regolare, FTW_D=directory, FTW_SL=symlink (con FTW_PHYS)
// ftwbuf->base = offset del nome nel path ; ftwbuf->level = profondita'
int cb(const char *fpath, const struct stat *sb, int typeflag, struct FTW *fb) {
    if (typeflag != FTW_F) return 0; // filtra solo i file
    printf("%s (nome: %s)\n", fpath, fpath + fb->base);
    return 0; // 0 = continua la visita
}
int main(int argc, char **argv) {
    // FTW_PHYS = non seguire i symlink (li riporta come FTW_SL)
    nftw(argv[1], cb, 20 /*max fd*/, FTW_PHYS);
}
```

Trucchi nftw: con FTW_ACTIONRETVAL puoi tornare FTW_SKIP_SUBTREE per saltare una sottocartella. La callback usa variabili *globali* per passare dati (file target, contatori, ecc.).

3.3 Python — os.walk (ricorsivo) e os.scandir (un livello)

```
# os.walk: visita ricorsiva, restituisce (dirpath, sottocartelle, file) per ogni nodo
for dirpath, dirnames, filenames in os.walk(root):
    for f in filenames:
        full = os.path.join(dirpath, f)
        print(full)

# os.scandir: un solo livello, ma con .stat().is_dir() veloci (niente syscall extra)
for entry in os.scandir(path):
    if entry.is_file(follow_symlinks=False): # equivalente a lstat+S_ISREG
        print(entry.path, entry.stat().st_ino)
```

Profondità in os.walk: depth = dirpath.replace(root, '').count(os.sep). Per ricorsione con limite di profondità conviene non os.scandir ricorsivo con contatore.

4 Controllare il tipo / i metadati di un file

4.1 C — stat vs lstat

- stat(path, &st): segue i symlink (riporta info sul *target*).
- lstat(path, &st): NON segue i symlink (riporta info sul *link stesso*). Usa lstat quando devi distinguere/contare i symlink.

```
struct stat st; lstat(path, &st);
if (S_ISREG(st.st_mode)) ... // file regolare
if (S_ISDIR(st.st_mode)) ... // directory
if (S_ISLNK(st.st_mode)) ... // symlink (solo con lstat)
// campi utili:
st.st_ino // numero di inode
st.st_dev // device (inode UNICO = coppia (st_dev, st_ino))
st.st_size // dimensione in byte
st.st_mtime // ultima modifica (time_t)
st.st_nlink // numero di hard link
st.st_mode // tipo + permessi ; bit permessi: S_IXUSR, S_IWGRP, ...
if (st.st_mode & S_IXUSR) ... // eseguibile dal proprietario?
```

4.2 C — riconoscere il contenuto dai "magic bytes"

```
int is_elf(const char *p){ FILE*f=fopen(p,"rb"); unsigned char b[4];
size_t n=fread(b,1,4,f); fclose(f);
return n==4 && b[0]==0x7f && b[1]=='E' && b[2]=='L' && b[3]=='F'; }
int is_script(const char *p){ FILE*f=fopen(p,"rb"); unsigned char b[2];
size_t n=fread(b,1,2,f); fclose(f);
return n==2 && b[0]=='#' && b[1]=='!'; } // shebang #!
```

4.3 Python — tipo e permessi

```
import os, stat
st = os.lstat(path)                # lstat: non segue symlink
stat.S_ISREG(st.st_mode)           # file?
stat.S_ISDIR(st.st_mode)           # dir?
stat.S_ISLNK(st.st_mode)           # symlink?
mode_str = stat.filemode(st.st_mode) # "-rwxr-xr-x" (come ls -l)
st.st_mode & stat.S_IXUSR           # eseguibile owner?
# scorciatoie su DirEntry di scandir:
entry.is_file(follow_symlinks=False); entry.is_dir(); entry.is_symlink()
```

5 Hard link, symlink, inode

5.1 Concetti

- **Hard link**: due nomi → stesso inode (stessa coppia st_dev+st_ino). Si crea con `link`. Identità file = (st_dev, st_ino) *non solo* l'inode.
- **Symlink**: file speciale che contiene un *path*. Si crea con `symlink`, si legge con `readlink`, si risolve con `realpath`.

5.2 C — creare link

```
link("orig", "nuovo_hardlink"); // hard link (stesso inode)
symlink("target_path", "nome_link"); // symlink (salva la stringa target)

// leggere dove punta un symlink (readlink NON mette il '\0!')
char buf[PATH_MAX];
ssize_t n = readlink(linkpath, buf, sizeof(buf)-1);
if (n != -1) buf[n] = '\0'; // terminare a mano = ERRORE TIPICO

// path assoluto/canonico (risolve symlink, "..", ".");
char *real = realpath(path, NULL); // malloc'd -> free dopo
// oppure su buffer fisso: realpath(path, abs_buf);
```

5.3 C — trovare hard link / symlink di un file (pattern d'esame classico)

```
// HARD LINK di F nel sottoalbero: stesso (dev,ino) ma path diverso dall'originale
if (sb->st_ino == F.st_ino && sb->st_dev == F.st_dev) {
    char *r = realpath(fpath, NULL);
    if (strcmp(r, F.realpath) != 0) printf("%s\n", fpath); // escludi originale
    free(r);
}
// SYMLINK che puntano a F: risolvi il target del link e confronta col realpath di F
if (typeflag == FTW_SL) {
    char buf[PATH_MAX]; ssize_t n = readlink(fpath, buf, sizeof(buf)-1);
    buf[n]='\0';
    char *rl = realpath(buf, NULL); // attenzione ai target RELATIVI
    if (rl && strcmp(rl, F.realpath)==0) printf("%s\n", fpath);
    free(rl);
}
```

Target relativi: se `readlink` ritorna un path che non inizia con `/`, va risolto rispetto alla directory del symlink (prendi dirname del link, concatena, poi `realpath`).

5.4 Python — link

```
os.symlink(target, linkname) # symlink
os.link(src, dst)            # hard link
os.readlink(linkname)        # legge il target
os.path.realpath(path)       # path canonico
os.makedirs(d, exist_ok=True) # mkdir -p (niente errore se esiste)
```

6 Creare / spostare / copiare directory e file

6.1 C — mkdir, rename (spostamento atomico), copia ricorsiva

```
// mkdir idempotente (ignora "esiste già")
if (mkdir(path, 0755) == -1 && errno != EEXIST) { perror("mkdir"); }

// SPOSTARE un file = rename (atomico, stesso filesystem)
rename("vecchio/path", "nuovo/path");

// COPIA RICORSIVA di un albero ricreando dir/symlink/hardlink:
int clone(const char *src, const char *dst){
    mkdir(dst, 0755);
```

```

DIR *d = opendir(src); struct dirent *e;
while ((e = readdir(d))) {
    if (!strcmp(e->d_name, ".") || !strcmp(e->d_name, "..")) continue;
    char from[PATH_MAX], to[PATH_MAX];
    snprintf(from, sizeof from, "%s/%s", src, e->d_name);
    snprintf(to, sizeof to, "%s/%s", dst, e->d_name);
    struct stat st; lstat(from, &st);
    if (S_ISDIR(st.st_mode)) clone(from, to); // ricorsione
    else if (S_ISLNK(st.st_mode)) { char t[PATH_MAX]; // ricrea symlink
        ssize_t n=readlink(from, t, sizeof t-1); t[n]='\0'; symlink(t, to); }
    else link(from, to); // hard link
}
closedir(d); return 0;
}

```

6.2 Python — alto livello

```

os.makedirs(d, exist_ok=True) # mkdir -p
os.rename(src, dst) # sposta (atomico)
import shutil
shutil.copy2(src, dst) # copia file + metadati
shutil.copytree(src, dst) # copia albero
shutil.move(src, dst) # sposta (anche tra filesystem)

```

7 Leggere / scrivere file e contenuto

7.1 C — leggere un intero file in memoria

```

FILE *f = fopen(path, "rb");
fseek(f, 0, SEEK_END); long sz = ftell(f); rewind(f);
char *buf = malloc(sz);
fread(buf, 1, sz, f); fclose(f);
// cercare una sottostringa binaria (GNU): memmem(buf, sz, needle, nlen)
if (memmem(buf, sz, target, strlen(target)) != NULL) printf("trovato\n");

```

7.2 C — digest SHA1 (OpenSSL, compila -lssl -lcrypto)

```

#include <openssl/sha.h>
void sha1_file(const char *path, char out[41]){
    FILE *f=fopen(path, "rb"); SHA_CTX c; SHA1_Init(&c);
    unsigned char b[65536]; size_t n;
    while ((n=fread(b,1,sizeof b,f))>0) SHA1_Update(&c,b,n);
    fclose(f);
    unsigned char d[SHA_DIGEST_LENGTH]; SHA1_Final(d,&c);
    for (int i=0;i<SHA_DIGEST_LENGTH;i++) sprintf(out+i*2,"%02x",d[i]);
    out[40]='\0';
}

```

7.3 Python — testo, encoding, digest

```

text = open(path, "r", encoding="utf-8").read()
open(out, "w", encoding="ascii").write(text) # forza ASCII
# carattere non-ASCII? ord(c) >= 128
def has_non_ascii(s): return any(ord(c) >= 128 for c in s)
import hashlib
h = hashlib.sha1(open(path, "rb").read()).hexdigest() # digest file

```

8 Argomenti, processi, exec

8.1 C — parametri e usage

```

int main(int argc, char **argv){
    if (argc != 3) { fprintf(stderr, "usage: %s <a> <b>\n", argv[0]); return 1; }
    // argv[0]=nome prog, argv[1]=primo arg ...
}
// scegliere modalita' da opzione:
if (strcmp(argv[1], "-l")==0) mode=HARD;
else if (strcmp(argv[1], "-s")==0) mode=SYM;

```

8.2 C — fork + exec + wait

```

pid_t pid = fork();
if (pid < 0) { perror("fork"); return 1; }
else if (pid == 0) { // FIGLIO
    execv(path, argv_array); // argv_array[0]=prog, ultimo = NULL !
    perror("execv"); exit(1); // si arriva qui solo se exec fallisce
} else { // PADRE
    int status; waitpid(pid, &status, 0);
    printf("exit code = %d\n", WEXITSTATUS(status));
}
// execv : path esatto + array argv ; execup : cerca nel $PATH

```

Regola d'oro: l'array per exec deve terminare con NULL, e argv[0] è il nome del programma.

8.3 C — leggere args da /proc (cmdline è separato da \0)

```
// /proc/PID/exe = symlink all'eseguibile (readlink)
// /proc/PID/cmdline = argomenti separati da '\0'
snprintf(p, sizeof p, "/proc/%s/cmdline", argv[1]);
int fd=open(p, O_RDONLY); int n=read(fd, cmd, sizeof cmd-1); close(fd); cmd[n]='\0';
char *args[100]; int k=0;
for (int i=0; i<n; i++) { args[k++] = cmd+i; i+=strlen(cmd+i)+1; } // split su '\0'
args[k]=NULL;
```

Lo stesso schema "buffer di stringhe separate da \0" si usa per leggere da stdin/pipe una lista di argomenti (vedi sezione FIFO).

9 Comunicazione tra processi (IPC)

9.1 C — FIFO / named pipe

```
// LETTORE
mkfifo(path, 0666);
int fd = open(path, O_RDONLY); // si BLOCCA finche' qualcuno apre in scrittura
char buf[PATH_MAX]; int pos=0, n;
while ((n=read(fd, buf+pos, sizeof buf-pos))>0) pos+=n; // leggi fino a EOF
// SCRITTORE: scrive ogni arg seguito da '\0', poi chiude
for (int i=1; i<argc; i++) { printf("%s", argv[i]); putchar('\0'); }
```

9.2 C — segnali real-time (passano un valore, sono in coda)

Schema: SIGRTMIN..SIGRTMAX sono affidabili e accodati; sigqueue invia un int (sival_int); il numero del segnale può codificare un indice.

```
// --- RICEVITORE ---
void handler(int sig, siginfo_t *info, void *ctx){
    int val = info->si_value.sival_int; // valore inviato
    int idx = sig - SIGRTMIN; // indice dal numero segnale
}
struct sigaction sa = {0};
sa.sa_sigaction = handler; sa.sa_flags = SA_SIGINFO; // SA_SIGINFO => 3 args
sigemptyset(&sa.sa_mask);
for (int i=0; i<=SIGRTMAX-SIGRTMIN; i++) sigaction(SIGRTMIN+i, &sa, NULL);
while(1) pause();
// --- TRASMETTITORE ---
union sigval sv; sv.sival_int = valore;
sigqueue(pid, SIGRTMIN + i, sv);
```

9.3 C — trasmettere bit a bit con SIGUSR1/SIGUSR2 + ack (variante d'esame)

Schema: ogni bit del carattere → un segnale (SIGUSR1=0, SIGUSR2=1). Dopo ogni blocco il ricevitore manda un ack al mittente (serve il pid del mittente: lo si legge da info->si_pid con SA_SIGINFO).

```
// --- RICEVITORE: ricostruisce un byte bit a bit ---
volatile sig_atomic_t cur=0, nbit=0, got_ack=0;
void on_bit(int sig, siginfo_t *info, void *c){
    int bit = (sig==SIGUSR2); // USR1->0, USR2->1
    cur |= (bit << nbit); // accumula (LSB first)
    if (++nbit==8){ putchar(cur); fflush(stdout); cur=0; nbit=0;
        kill(info->si_pid, SIGUSR1); // ACK al mittente (pid dal segnale)
    }
}
void on_ack(int s){ got_ack=1; }
// --- MITTENTE: invia un byte bit a bit, poi aspetta l'ack ---
for (int b=0; b<8; b++){
    kill(rx_pid, (c>>b)&1 ? SIGUSR2 : SIGUSR1);
    usleep(2000); // "rallentare ad arte" -> evita perdite
}
while(!got_ack) pause(); got_ack=0; // handshake prima del prossimo byte
```

Nota: SIGUSR1/SIGUSR2 non sono accodati (a differenza dei real-time): vanno scanditi/serializzati, da qui l'ack e gli usleep. Mandare anche il terminatore '\0' per segnalare fine stringa.

9.4 C — contare segnali con signalfd (variante d'esame)

Alternativa a sigaction: i segnali diventano leggibili da un file descriptor (niente handler asincrono).

```
#include <sys/signalfd.h>
sigset_t m; sigemptyset(&m); sigaddset(&m, SIGUSR1); sigaddset(&m, SIGUSR2);
sigprocmask(SIG_BLOCK, &m, NULL); // OBBLIGATORIO: blocca la consegna normale
int fd = signalfd(-1, &m, 0);
struct signalfd_siginfo si;
while (read(fd, &si, sizeof si) == sizeof si) {
    if (si.ssi_signo == SIGUSR1) c1++;
    else if (si.ssi_signo == SIGUSR2) c2++;
}
```

10 inotify — reagire a eventi del filesystem

```
#include <sys/inotify.h>
```

```

#define MASK (IN_CLOSE_WRITE | IN_MOVED_TO) // file finito di scrivere / spostato dentro
int fd = inotify_init();
inotify_add_watch(fd, dir, MASK);
char buf[4096];
while (1) {
    ssize_t n = read(fd, buf, sizeof buf); // BLOCCA fino a un evento
    struct inotify_event *ev = (void*)buf;
    while ((char*)ev < buf + n) {
        if (ev->mask & MASK) { /* ev->name = nome file coinvolto */ }
        // avanzare: dimensione struct + lunghezza nome variabile
        ev = (void*)((char*)ev + sizeof(struct inotify_event) + ev->len);
    }
}

```

Maschere comuni: IN_CREATE, IN_DELETE, IN_MODIFY, IN_CLOSE_WRITE, IN_MOVED_TO, IN_MOVED_FROM.

11 Pattern Python ricorrenti

11.1 Raggruppare file per criterio + creare cartelle/symlink

```

from collections import defaultdict
import os, stat
by_key = defaultdict(list)
for entry in os.scandir(dir):
    if not entry.is_file(follow_symlinks=False): continue
    key = stat.filemode(os.stat(entry.path).st_mode) # es: raggruppa per permessi
    by_key[key].append(entry.path)
for key, paths in by_key.items():
    os.makedirs(key, exist_ok=True)
    for p in paths:
        os.symlink(p, os.path.join(key, os.path.basename(p)))

```

11.2 Ordinare i file per profondità / nome / inode

```

files = []
for dirpath, dirs, names in os.walk(root):
    depth = dirpath.replace(root, '').count(os.sep)
    for f in names: files.append((depth, f, os.path.join(dirpath, f)))
# piu' profondi prima, a parita' ordine alfabetico
files.sort(key=lambda x: (-x[0], x[1]))

```

11.3 Tabella inode con profondità limitata (scandir ricorsivo)

```

items = []
def walk(folder, remaining):
    if remaining == -1: return
    for e in os.scandir(folder):
        items.append((e.stat().st_ino, e.path))
        if e.is_dir(): walk(e.path, remaining-1)
items.sort(key=lambda x: x[0]) # ordina per inode

```

11.4 Aggiungere testo come commento in base al linguaggio (per suffisso)

```

import os
def banner(text, ext):
    if ext == ".c": return f"/*\n{text}\n*/\n"
    if ext == ".py": return f"'''\n{text}\n'''\n"
    if ext == ".sh": return f"'\n{text}\n'\n"
for f in os.listdir("."):
    ext = os.path.splitext(f)[1]
    if ext not in (".c", ".sh", ".py"): continue
    old = open(f).read()
    if ext == ".sh" and old.startswith("#!"): # bash: dopo lo shebang
        first, _, rest = old.partition("\n")
        open(f, "w").write(first + "\n" + banner(MSG, ext) + rest)
    else:
        open(f, "w").write(banner(MSG, ext) + old)

```

11.5 Confronto contenuto file / eliminare duplicati di f (dremcont)

```

import os, hashlib
def sha1(p): return hashlib.sha1(open(p, "rb").read()).hexdigest()
target = sha1(f)
for dp, _, names in os.walk(d):
    for n in names:
        p = os.path.join(dp, n)
        if os.path.isfile(p) and sha1(p) == target:
            os.remove(p) # stesso contenuto -> cancella

```

12 Errori tipici da NON fare (checklist mentale)

- Dimenticare `buf[n]='\0'` dopo `readlink/read` (non terminano la stringa).
- Non saltare `"."` e `".."` in `readdir` → ricorsione infinita.
- Usare `stat` invece di `lstat` quando devi distinguere/contare i symlink.

- Identificare un file solo per `st_ino`: serve la coppia (`st_dev`, `st_ino`).
- Dimenticare `NULL` finale nell'array di `exec`, o `argv[0]`.
- Mancata feature-test macro → `nftw/memmem` "implicit declaration".
- Buffer dei path troppo piccolo: usa `PATH_MAX` e `snprintf` (non `sprintf`).
- Symlink con target relativo: vanno risolti rispetto alla dir del link.
- Non gestire `EEXIST` su `mkdir/symlink` quando l'oggetto può già esistere.
- Compilazione: `gcc -Wall -o prog prog.c` ; con OpenSSL aggiungi `-lssl -lcrypto`.